



Azure Migration Pack

Everything required to move the TAQA Knowledge Hub onto TAQA Azure: what the platform is, the resources needed, and the step-by-step transition.

Prepared for	Rafa Al Zahrani, D&T
Prepared by	Mohammed Al-Jahdali
Department	TAQA Learning Center
Platform	TAQA Knowledge Hub

INTERNAL USE

1. What the website actually is

It is a folder of files. That is the whole thing.

31 files, 1.9 MB. No database, no server, no backend. No `npm install`, no build step, no compiling. The files sitting in the repository are exactly the files served to the browser.

This is the most important fact for planning, because it means there is nothing to migrate except static files. No data to export, no server to rebuild, no dependencies to resolve.

What is in the folder

Type	Files	What they are
Pages	11 <code>.html</code>	Home, dashboard, segment view, document library, document viewer, search, analytics, glossary, upload, support ticket, offline page
Scripts	4 <code>.js</code>	Shared navigation and UI, two data files holding document titles, and the service worker
Fonts	2 <code>.otf</code>	The TAQA corporate typeface, Bw Gradual, self-hosted at 144 KB
Images	5 plus icons	Hero images and app icons, 868 KB
Config	<code>manifest.json</code> , <code>staticwebapp.config.json</code>	App install settings and Azure routing rules

What it does today

Staff open it and browse documents organised by the 12 TAQA segments, including Coiled Tubing, Drilling, QHSE and Cybersecurity. They can search across segments, read a glossary, bookmark items and raise a support ticket. It works offline and installs on a phone like an app.

What it does not do today

This is the honest part, and it matters more than the feature list.

There is no login, so the site is open to anyone with the URL. There are no real documents either: the titles are text typed into a JavaScript file, and the viewer shows a placeholder image rather than the actual PDF. Nothing is saved centrally, so bookmarks and notifications live in one browser and are neither shared nor backed up.

The front of the platform is complete and proven. The Azure move is what turns it into a real, governed TAQA system.

2. What you need from Azure

The footprint is deliberately small, because this is a static site.

Required now, to host it

#	What	Notes
1	Resource group in the TAQA tenant	With a cost centre assigned
2	Azure Static Web App resource	Standard tier, needed for Entra ID auth and custom domains. Free tier works for a pilot but carries no SLA
3	Deployment token stored as a repository secret	Azure issues this when the resource is created
4	Repository under the TAQA organisation	GitHub Enterprise or Azure DevOps, your call. Code moves with full history
5	A URL	Either the Azure-generated address or a TAQA custom domain, your preference

Required for login, Phase 1

#	What	Notes
6	Entra ID App Registration	The main blocker. Nothing about login can start without it
7	Client ID and Tenant ID	Issued by the App Registration
8	Client secret, stored in the Static Web App settings	Never in the code
9	Redirect URI registered	<a href="https://<your-final-url>/.auth/login/aad/callback">https://<your-final-url>/.auth/login/aad/callback
10	Graph permission <code>User.Read</code> with admin consent	The minimum for sign-in. SharePoint scopes come later, in Phase 2

Not needed

No virtual machines, no App Service, no database, no storage account, no Application Gateway. If a quote comes back including any of those, the scope has been misread.

3. Step-by-step migration

Six steps. Steps 1 to 4 are the move, and steps 5 and 6 add login.

Step 1: take the code

I transfer the repository into the TAQA organisation, or you fork it, whichever matches D&T policy. Full commit history comes across intact, which preserves the audit trail from the Secure SDLC review already completed.

You receive the repository, this document, and the handover plan in [docs/](#).

Step 2: create the Azure resource

Create the Static Web App in the TAQA resource group and link it to the repository. Azure generates the deployment workflow automatically.

One setting trips people up, because there is no build step:

```
app_location:    "/"
api_location:    ""
output_location: ""    <-- must be EMPTY
```

If `output_location` is set to `dist` or `build`, the deploy fails looking for a folder that does not exist. There is no build output. The source is the site.

Step 3: deploy and verify

Push to `main` and the site deploys in about two minutes.

Check five things. The home page should load with the TAQA typeface rather than a fallback. Navigation between pages should work, search should return results, and the Arabic toggle should switch the layout right to left. Finally, switch the network off and confirm the site still loads.

`staticwebapp.config.json` is already in the repository and handles routing, font MIME types, caching and security headers. You do not need to configure it.

Step 4: lock the code down

Turn on branch protection on `main`: no direct pushes, and a pull request plus one approval required.

Nominate the D&T technical reviewer as approver.

This satisfies the code review control from the Kosli audit, and it is the point at which the platform comes formally under D&T governance.

At this stage the migration is done. The site runs on TAQA Azure, under TAQA source control, with a review gate. Everything below adds login.

Step 5: add Entra ID sign-in

Create the App Registration, store the client secret in the Static Web App configuration, then add this block to `staticwebapp.config.json`:

```

"auth": {
  "identityProviders": {
    "azureActiveDirectory": {
      "registration": {
        "openIdIssuer": "https://login.microsoftonline.com/<TENANT_ID>/v2.0",
        "clientIdSettingName": "AAD_CLIENT_ID",
        "clientSecretSettingName": "AAD_CLIENT_SECRET"
      }
    }
  }
},
"routes": [
  { "route": "/*", "allowedRoles": ["authenticated"] }
]

```

Sequence matters here. Add this after the App Registration exists. Applying it first locks everyone out of a site that has no way to let them back in.

Azure Static Web Apps handles the sign-in flow natively, so no authentication code needs writing. The logged-in user's name and email become available to the pages automatically at `/.auth/me`.

Step 6: decide who gets in

Two options, your call with the business. Either open it to all TAQA staff, so anyone with a TAQA account can sign in, which is the simplest route. Or restrict by segment using Entra security groups, so users see only their own, which means more setup and more ongoing administration.

My recommendation is to open it to all staff at launch. The content is internal reference material, and per-segment restriction can be added later once real usage shows whether it is actually needed.

4. External dependencies, already removed

The site was built as a standalone prototype and originally called four outside services. All four have been removed. They are noted here so the history is visible if security review asks.

What it was	Where it sent data	What replaced it
Sign-up form posting name and email	Google Apps Script	The external call is deleted. The name is kept in the user's own browser only, and Entra ID sign-in replaces the form entirely at step 5
Page analytics	Google Analytics and Tag Manager	Removed completely. The replacement should be a D&T-approved tool
QR code generation	api.qrserver.com	Generated inside the browser using a bundled MIT-licensed library. The generated codes were decoded and verified to point at the correct URL
Web fonts	Google Fonts CDN	Urbanist and Inter are now self-hosted at 398 KB, Latin subset. Typography is unchanged

The site now makes no outbound requests to any third party. This was verified by loading all ten pages in a browser and recording every network request, and the only traffic went to the site's own origin.

The Content-Security-Policy on every page was tightened to match:

```
default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; font-src 'self' data;;
img-src 'self' data: blob;; connect-src 'self';
worker-src 'self'; manifest-src 'self'
```

Anything that tries to call a third party is now blocked by the browser itself, rather than simply being absent from the code.

One item left, and it is your decision

The site currently deploys to GitHub Pages as well as Azure, which means a second public copy outside TAQA hosting. I have left that workflow in place rather than switching it off on my own, since it is the live URL people may be using today. Say the word at handover and it goes.

5. What I need from you

1. Confirm GitHub Enterprise or Azure DevOps for the repository
2. Confirm Standard or Free tier for the Static Web App
3. Confirm the URL, either Azure default or a TAQA custom domain
4. Tell me who the technical reviewer is, so I can add them
5. Start the App Registration request. It has the longest lead time, so it is worth beginning even while the other items are being decided

6. Realistic timeline

Stage	Effort	Depends on
Steps 1 to 4, hosted on TAQA Azure	1 to 2 days	Resource group and repository decision
Steps 5 and 6, Entra ID login live	2 to 3 days	App Registration approval

The work itself is small. The waiting is on approvals rather than engineering.

Mohammed Al-Jahdali. mohammed.jahdali@tq.com. +966 54 773 3744 Happy to walk through the codebase whenever suits you.
